

Ejercicio 2 - Arreglos básicos

Control de Tiempos en Torneo Universitario de Atletismo

Objetivo

- Aplicar arreglos básicos para almacenar información dentro de un programa.
- Acceder y modificar elementos de un arreglo mediante índices.
- Recorrer arreglos utilizando estructuras repetitivas.
- Realizar operaciones de búsqueda y cálculo utilizando los datos almacenados en un arreglo.
- Aplicar de forma correcta los modificadores de visibilidad para no violar el encapsulamiento.
- Modelar el programa utilizando UML.
- Crear un programa principal que resuelva el problema utilizando programación orientada a objetos.

Introducción

La Universidad del Valle de Guatemala organizará un torneo universitario de atletismo y desea implementar un sistema sencillo para registrar los tiempos obtenidos por los participantes durante diferentes pruebas de clasificación.

Cada atleta podrá realizar varios intentos para mejorar su tiempo. Los organizadores necesitan almacenar estos resultados para consultar el desempeño del participante y determinar cuál ha sido su mejor resultado durante la competencia.

Como primera versión del sistema, se trabajará con un único atleta activo y se podrán registrar como máximo 10 tiempos correspondientes a sus diferentes intentos.

Para almacenar los tiempos obtenidos durante la competencia se deberán utilizar arreglos básicos.

Para administrar los tiempos del atleta se debe realizar lo siguiente:

1. Al iniciar el programa se debe solicitar la información del atleta. Como mínimo se debe ingresar:
 - a. Nombre.
 - b. Número de participante.
 - c. Edad.
2. Cada atleta podrá realizar un máximo de 10 intentos. Los tiempos obtenidos deberán almacenarse utilizando un arreglo.
3. Al registrar un nuevo intento se deberá ingresar el tiempo obtenido por el atleta en segundos.
4. Antes de almacenar el tiempo se deberá validar que:
 - a. El tiempo ingresado sea mayor que 0.
 - b. Todavía exista espacio disponible dentro del arreglo.
5. Cada nuevo tiempo válido deberá almacenarse en la siguiente posición disponible del

arreglo.

6. El sistema deberá permitir consultar todos los tiempos registrados hasta el momento. No deberán mostrarse como intentos realizados aquellas posiciones del arreglo que todavía no hayan sido utilizadas.
7. El sistema deberá permitir consultar el tiempo correspondiente a un intento específico. Para esto, el usuario deberá indicar el número de intento que desea consultar.
8. El sistema deberá permitir modificar el tiempo de un intento previamente registrado. Para realizar la modificación:
 - a. Se solicitará el número de intento.
 - b. Se solicitará el nuevo tiempo.
 - c. El nuevo tiempo deberá ser mayor que 0.
 - d. No se podrá modificar un intento que todavía no haya sido registrado.
9. El sistema deberá calcular el tiempo promedio obtenido por el atleta. Para el cálculo únicamente deberán considerarse los intentos registrados.
10. El sistema deberá determinar cuál ha sido el mejor tiempo obtenido por el atleta. Debido a que se trabaja con tiempos de carrera, el valor más bajo registrado representa el mejor resultado.
11. El sistema deberá determinar cuál ha sido el tiempo más alto registrado por el atleta.
12. El sistema deberá indicar cuántos intentos ha realizado el participante y cuántos intentos tiene disponibles de los 10 permitidos.

Ejercicio

Análisis

Responda las siguientes preguntas:

1. ¿Qué propiedades y métodos tendrá cada clase?
2. ¿Qué tipo deben tener las propiedades y métodos de cada clase?
3. ¿Cuál de las propiedades identificadas debe implementarse utilizando un arreglo? ¿Qué tipo de datos almacenará?
4. ¿Cuáles deben ser los modificadores de visibilidad de los miembros en cada clase?
5. ¿Qué parámetros serán requeridos por los métodos en sus clases?
6. ¿Cómo proveerá de valores iniciales a sus objetos? ¿Qué valores iniciales les asignará?
7. ¿Cómo determinará cuál es la siguiente posición disponible dentro del arreglo?
8. ¿Cómo recorrerá únicamente las posiciones del arreglo que contienen tiempos registrados?

Diseño

Desarrolle un diagrama de clases que muestre los miembros de cada clase y que establezca las relaciones necesarias entre ellas.

Incluya en su diagrama al driver program, que contiene el método main().

El diagrama deberá representar correctamente los atributos, métodos, tipos de datos, parámetros y modificadores de visibilidad identificados durante el análisis.

Use anotaciones (recuadros de texto) en su diagrama para aclarar cualquier información que considere necesaria.

Programa

Desarrolle un sistema que contenga un driver program y las instancias necesarias de las clases identificadas durante su análisis.

El programa deberá utilizar como mínimo un arreglo básico para almacenar los tiempos registrados por el atleta.

Así mismo, debe mostrar un menú con las siguientes opciones:

1. Nuevo atleta: reemplaza la instancia actual del atleta activo. Al crear un nuevo atleta, este comienza sin intentos registrados.
2. Registrar nuevo intento: solicita el tiempo obtenido por el atleta y lo almacena en la siguiente posición disponible del arreglo.
3. Consultar tiempos: muestra todos los tiempos registrados por el atleta hasta el momento.
4. Consultar un intento: solicita el número de intento y muestra el tiempo correspondiente.
5. Modificar tiempo: solicita el número de un intento previamente registrado y permite modificar el tiempo almacenado.
6. Mostrar promedio: calcula y muestra el promedio de los tiempos registrados.
7. Mostrar mejor y mayor tiempo: muestra el mejor tiempo obtenido y el tiempo más alto registrado por el atleta.
8. Consultar intentos disponibles: muestra cuántos intentos se han registrado y cuántas posiciones continúan disponibles de las 10 permitidas.
9. Salir: termina la ejecución del programa.

El programa deberá validar las entradas necesarias para evitar almacenar tiempos inválidos o acceder a posiciones que no correspondan a intentos previamente registrados.

Material a entregar en Canvas

- **Archivo PDF con el análisis y diseño.**
 - No incluya únicamente la URL del diagrama; adjunte el diagrama dentro del PDF o como un archivo externo.
 - Use la plantilla proporcionada.
- **URL al repositorio público en GitHub** conteniendo todos los archivos .java necesarios para ejecutar el programa.

Evaluación

- **Análisis** (30 puntos)
 - **(5 puntos)** Se listan correctamente todos los requisitos funcionales del sistema a desarrollar, punto por punto.
 - **(5 puntos)** Se describe el propósito de cada una de las clases identificadas. La descripción es lógica y coherente con lo que hace la clase dentro del sistema que modela y resuelve el problema presentado.
 - **(10 puntos)** Se describe el propósito de cada atributo de cada clase. La descripción es suficiente para comprender la razón de la creación del atributo y su importancia como propiedad de las instancias o clases. Se identifica correctamente el atributo que utiliza un arreglo.
 - **(10 puntos)** Se describe el propósito de la creación de cada método de cada clase. La descripción es suficiente para comprender qué acciones realizará cada método, qué información requerirá para funcionar y qué resultados producirá.
- **Diseño** (30 puntos)
 - **(8 puntos)** La representación de las clases coincide con el estándar del lenguaje UML.
 - **(12 puntos)** Están representados cada uno de los componentes de las clases de manera correcta y coinciden con lo descrito en el análisis.
 - **(10 puntos)** Las relaciones entre las clases están correctamente representadas.
- **Programa** (40 puntos)
 - **(10 puntos)** La cantidad de clases identificadas es suficiente para resolver el problema planteado y se utilizan correctamente los principios básicos de programación orientada a objetos.
 - **(15 puntos)** Se implementa correctamente un arreglo para almacenar los tiempos y se realizan las operaciones solicitadas sobre sus elementos utilizando índices y estructuras repetitivas.
 - **(10 puntos)** El programa permite registrar, consultar y modificar tiempos, así como calcular correctamente el promedio, el mejor tiempo y el tiempo más alto.
 - **(5 puntos)** El programa valida correctamente los límites del arreglo, las posiciones utilizadas y los valores permitidos, evitando accesos inválidos.